

**SECJ4383**

**SOFTWARE CONSTRUCTION**

# **DESIGN OF CODE, CODE SMELLS DETECTION & REFACTORING**

*A technical assessment of TVPSSHub*

Prepared by: Project Team

Assessment scope: Tasks B1-B5

Date: 2 July 2026

**DOCUMENT STATUS** Submission-ready analytical draft. Genuine runtime screenshots and the final refactored repository/commit evidence must be inserted after implementation; no evidence has been invented.

**REPORT OVERVIEW**

## Executive Summary

TVPSSTHub is a Java 11 Spring Boot 2.7.18 web application for managing users, schools, activities, educational resources, and feedback in a school media-program context. The production Java source contains 24 files and 1,916 non-blank, non-comment lines, comfortably exceeding the 500 LOC requirement. The review concentrated on the controller layer because it contains the highest concentration of orchestration, validation, authorization, pagination, and view-model preparation.

The manual review identified four recognized smell categories: God Class/Large Class, Long Method, Duplicate Code, and Dead Code. The two primary smells selected for refactoring are the God Class in `UserController` and duplicated pagination/authorization logic across controller methods. PMD 7.17.0 independently confirmed 12 issues, including 11 unused imports (dead code) and one misplaced null check; CPD reported no token-identical clone above its default threshold.

### Key recommendations

- Extract focused registration, dashboard, profile, and teacher-user management responsibilities from `UserController`.
- Extract reusable pagination and current-user/access-check helpers, keeping controllers thin and behaviorally equivalent.
- Remove PMD-confirmed unused imports and correct the `ResourceController` null-check ordering as a low-risk cleanup.
- Add automated controller/service tests before moving code; the current Maven test lifecycle passes but no test classes are present.

**ASSESSMENT CONCLUSION** The codebase is suitable for smell analysis and refactoring, but B5 cannot be truthfully marked complete until the proposed changes are committed under `/refactored-code` and the affected features are rerun.

## NAVIGATION

## Table of Contents

1. Introduction and Scope
2. Task B1 - Codebase Selection and Documentation
3. Architecture and Baseline Metrics
4. Task B2 - Code Smell Identification
5. Smell 1 - God Class / Large Class
6. Smell 2 - Duplicate Code
7. Additional Findings - Long Method and Dead Code
8. Task B3 - Detection Methodology
9. Automated Analysis Evidence
10. Task B4 - Refactoring Strategy
11. Refactoring Design: Extract Class
12. Refactoring Design: Extract Method / Shared Component
13. Task B5 - Before/After Comparison
14. Verification Plan and Current Results
15. Risk, Regression and Maintainability Assessment
16. Submission Checklist
17. References and Appendices

*Note: Page numbering is generated by Word/LibreOffice. Update the field if the document is edited before submission.*

## CONTEXT

# 1. Introduction and Scope

This report evaluates the design quality of TVPSSHub against the Part B rubric for codebase selection, smell identification, manual and automated detection, refactoring selection, application evidence, and behavioral verification. The analysis is intentionally evidence-led: every file and line reference maps to the repository state reviewed on 2 July 2026.

## Objectives

- Verify that the selected source exceeds 500 executable/source LOC.
- Identify at least two distinct smells from the recognized categories and explain their concrete impact.
- Cross-reference manual review with PMD and CPD output.
- Propose Fowler-aligned refactorings that preserve routes, views, persistence behavior, and access rules.
- Define credible post-refactoring checks and clearly separate completed evidence from pending evidence.

## Scope boundaries

Primary scope: `src/main/java`, with supporting review of `pom.xml` and templates where needed to understand controller responsibilities. Generated classes under `target/classes`, SQL dumps, CSS, images, and duplicated deployed templates were excluded from source LOC. No claim is made that the proposed refactoring has already been applied; documentation alone cannot substitute for the required `/original-code` and `/refactored-code` repository structure.

## B1

## 2. Task B1 - Codebase Selection and Documentation

### System description (3-5 sentences)

TVPSSTHub is a web-based hub for coordinating school users, institutions, media-related activities, educational resource requests, and activity feedback. Its main features include registration and role-based login, user/profile administration, school records, activity creation and filtering, resource-request workflows, and feedback capture. The application is implemented in Java 11 with Spring Boot 2.7.18, Spring MVC, Spring Security, Thymeleaf/JSP views, Hibernate, Maven, and MySQL. It is intended for administrators, teachers, and students participating in a school media or TVPSS program. The project is group-developed coursework; therefore, no external source-code URL, third-party author, or external code licence applies to the codebase itself.

| Selection criterion     | Evidence                                                         | Result          |
|-------------------------|------------------------------------------------------------------|-----------------|
| Minimum 500 LOC         | 1,916 Java SLOC; blank/comment-only lines excluded               | Pass            |
| Structured/OOP language | Java 11 classes, controllers, services, DAOs and models          | Pass            |
| Suitable source         | Group-developed TVPSSTHub coursework system                      | Pass            |
| Repository packaging    | Current root contains source; rubric folders are not yet present | Action required |

### Required repository arrangement

```

/original-code    # immutable baseline used for this report
/refactored-code  # behavior-preserving revised implementation
/docs            # final PDF and evidence images (recommended)

```

## B1 EVIDENCE

### 3. Architecture and Baseline Metrics

| Layer      | Representative components              | Responsibility                                |
|------------|----------------------------------------|-----------------------------------------------|
| Controller | UserViewController, ActivityController | HTTP routes, authorization, model preparation |
| Service    | UserService, ActivityService           | Application-facing operations                 |
| DAO        | UserDAO, ActivityDAO                   | Hibernate queries and transactions            |
| Model      | UserViewModel, School, Resource        | Domain and form data                          |
| View       | Thymeleaf/JSP templates                | Server-rendered user interface                |

#### Largest Java files

| Rank | File                                                         | SLOC |
|------|--------------------------------------------------------------|------|
| 1    | src/main/java/com/example/controller/UserViewController.java | 257  |
| 2    | src/main/java/com/example/controller/ActivityController.java | 161  |
| 3    | src/main/java/com/example/dao/UserDAO.java                   | 159  |
| 4    | src/main/java/com/example/model/ActivityViewModel.java       | 144  |
| 5    | src/main/java/com/example/model/School.java                  | 136  |
| 6    | src/main/java/com/example/controller/ResourceController.java | 111  |
| 7    | src/main/java/com/example/dao/ActivityDAO.java               | 105  |
| 8    | src/main/java/com/example/dao/ResourceDAO.java               | 97   |
| 9    | src/main/java/com/example/config/SecurityConfig.java         | 83   |
| 10   | src/main/java/com/example/model/UserViewModel.java           | 78   |

Interpretation: UserViewController is the largest orchestration class by a clear margin. Size alone is not proof of a smell, but the class also has five collaborators and spans several unrelated user-facing use cases, which supports the Large Class classification.

## B2 SUMMARY

## 4. Task B2 - Code Smell Identification

| No. | Smell type              | Location                                                               | Specific evidence                                                                                                                     |
|-----|-------------------------|------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|
| 1   | God Class / Large Class | UserViewController.java, lines 22-353                                  | Five injected collaborators and 14 route handlers covering registration, dashboard, profile, listing, CRUD, authorization and logout. |
| 2   | Duplicate Code          | UserViewController.java 192-236 and<br>ActivityController.java 135-172 | Near-identical pagination workflow: page size, counts, range clamping, start/end calculation, subList and model attributes.           |
| 3   | Long Method             | UserViewController.java 49-94 and<br>192-236                           | Each handler is about 45 lines and combines<br>validation/querying/mutation or filtering/pagination/view preparation.                 |
| 4   | Dead Code               | Multiple imports; exact PMD locations<br>in Section 7                  | PMD reports nine unused imports, including UserViewController lines<br>13-14.                                                         |

**PRIMARY REFACTORING SCOPE** Smells 1 and 2 are treated as the minimum two distinct categories. Smells 3 and 4 demonstrate systematic review beyond the minimum.

## B2 DETAILED ANALYSIS

## 5. Smell 1 - God Class / Large Class

UserViewController is classified as a God Class/Large Class because it acts as the central coordinator for too many unrelated user concerns. The class knows about users, activities, schools, resources, password encoding, authentication, role restrictions, school tenancy, filtering, pagination, and view names.

| Responsibility      | Methods / lines                                         | Why separate                                              |
|---------------------|---------------------------------------------------------|-----------------------------------------------------------|
| Registration        | showRegisterForm; processRegisterForm (40-94)           | Validation and account creation form a cohesive use case. |
| Dashboard           | showDashboard (102-135)                                 | Aggregates three domains and diagnostic logging.          |
| Profile             | showProfile; showUpdateProfile; updateProfile (137-187) | Self-service profile concern.                             |
| Teacher user admin  | showUserList through deleteUser (189-344)               | School-scoped user management and authorization.          |
| Authentication view | login/logout (96-100, 346-350)                          | Navigation/security concern.                              |

### Maintenance impact

- A change to user validation, dashboard content, or teacher authorization all modifies the same class.
- The five injected dependencies broaden the class's reasons to change and its testing setup.
- Reviewers must understand unrelated flows before making a local modification.
- Future growth will likely worsen merge conflicts and regression risk.



## B2 DETAILED ANALYSIS

## 6. Smell 2 - Duplicate Code

The user-list and feedback-list handlers implement the same pagination algorithm with only the element type, empty-list behavior, and model attribute names changed. This is semantic duplication even though PMD CPD does not report a clone at its default token threshold.

| Step      | User list (192-236)          | Feedback list (135-172)      |
|-----------|------------------------------|------------------------------|
| Page size | 10                           | 10                           |
| Count     | userList.size()              | feedbackList.size()          |
| Pages     | ceil(totalItems/pageSize)    | ceil(totalItems/pageSize)    |
| Bounds    | Clamp page to 1..totalPages  | Clamp page to 1..totalPages  |
| Slice     | subList(start,end)           | subList(start,end)           |
| Model     | items,currentPage,totalPages | items,currentPage,totalPages |

### Secondary authorization duplication

UserViewController repeatedly resolves the authenticated email to a user (lines 199, 242, 253, 280-281, 305-306, and 322-323), then repeats school-membership checks in edit and delete flows. The repeated pattern risks inconsistent null handling and access messages.

**CPD INTERPRETATION** A negative clone-detector result does not invalidate a manual duplicate-code finding. Token thresholds are conservative; reviewers must still detect repeated algorithms and policy logic expressed with different identifiers.

## SYSTEMATIC REVIEW

## 7. Additional Findings - Long Method and Dead Code

### Long Method

processRegisterForm (lines 49-94) performs field validation, password confirmation, duplicate email checking, identity-card uniqueness checking, password encoding, role assignment, persistence, exception handling, and navigation. showUserList (lines 192-236) combines authentication context, filtering, pagination, page correction, slicing, and model preparation. Both are readable in isolation, but their mixed responsibilities make extension and unit testing harder.

### Dead Code confirmed by PMD

| File / line                | Unused import     |
|----------------------------|-------------------|
| UserViewController.java:13 | HttpSession       |
| UserViewController.java:14 | LocalDate         |
| ActivityDAO.java:9         | SessionFactory    |
| ActivityDAO.java:10        | Autowired         |
| ActivityDAO.java:12        | Hibernate         |
| FeedbackDAO.java:3         | ActivityViewModel |
| FeedbackDAO.java:11        | SessionFactory    |
| FeedbackDAO.java:13        | Autowired         |
| ActivityViewModel.java:5-6 | List, ArrayList   |
| UserViewModel.java:5       | DateTimeFormat    |

PMD produced 11 unused-import entries (plus a separate misplaced-null-check finding), making dead code the clearest directly automated smell. Removing these imports is behavior-neutral and reduces noise.

## B3 MANUAL DETECTION

## 8. Task B3 - Detection Methodology

### Manual review procedure

- Enumerate Java source files and exclude generated output, blank lines, and comment-only lines from LOC.
- Rank classes by SLOC to identify unusually large coordination points.
- Map each controller method to a business responsibility and count injected collaborators.
- Inspect long handlers for mixed responsibilities, branches, mutation, persistence, and view preparation.
- Compare repeated list-processing and access-control sequences across controllers.
- Record precise file/line ranges, then validate independent symptoms with PMD and CPD.

### Warning signs and thresholds

| Indicator          | Observed value                     | Reasoning                                                             |
|--------------------|------------------------------------|-----------------------------------------------------------------------|
| Class size         | UserController: 353 physical lines | >300 lines plus broad responsibility map warrants Large Class review. |
| Collaborators      | 5 injected fields                  | High fan-out for one MVC controller.                                  |
| Route handlers     | 14                                 | Unrelated feature families coexist.                                   |
| Method length      | ~45 lines for two handlers         | Not automatically wrong, but mixed concerns confirm Long Method.      |
| Repeated algorithm | Two pagination blocks              | Same change would require edits in two controllers.                   |

Cyclomatic complexity was estimated structurally from decision points during review but is not reported as a tool-generated exact metric because PMD's default report did not emit per-method complexity values. This avoids overstating evidence.

## B3 AUTOMATED DETECTION

## 9. Automated Analysis Evidence

### PMD static-analysis report

```

PMD 7.17.0 | Java analysis completed 2026-07-02
ResourceController.java:106 MisplacedNullCheck - The null check here is misplaced; if the
variable 'auth' is null there will be a NullPointerException
UserController.java:13 UnnecessaryImport - Unused import
'javax.servlet.http.HttpSession'
UserController.java:14 UnnecessaryImport - Unused import 'java.time.LocalDate'
ActivityDAO.java:9 UnnecessaryImport - Unused import 'org.hibernate.SessionFactory'
ActivityDAO.java:10 UnnecessaryImport - Unused import
'org.springframework.beans.factory.annotation.Autowired'
ActivityDAO.java:12 UnnecessaryImport - Unused import 'org.hibernate.Hibernate'
FeedbackDAO.java:3 UnnecessaryImport - Unused import
'com.example.model.ActivityViewModel'
FeedbackDAO.java:11 UnnecessaryImport - Unused import 'org.hibernate.SessionFactory'
FeedbackDAO.java:13 UnnecessaryImport - Unused import
'org.springframework.beans.factory.annotation.Autowired'
ActivityViewModel.java:5 UnnecessaryImport - Unused import 'java.util.List'
ActivityViewModel.java:6 UnnecessaryImport - Unused import 'java.util.ArrayList'

```

Figure 1. PMD 7.17.0 report excerpt generated from target/pmd.xml.

| Tool                     | Command                    | Outcome                                                        |
|--------------------------|----------------------------|----------------------------------------------------------------|
| PMD 7.17.0               | mvn -q -DskipTests pmd:pmd | 12 violations: 11 unused imports and one misplaced null check. |
| PMD CPD 7.17.0           | mvn -q -DskipTests pmd:cpd | No duplication block exceeded the default threshold.           |
| Maven Surefire lifecycle | mvn -q test                | Exit code 0; no test classes exist.                            |

Cross-reference: PMD directly supports the Dead Code finding, while the null-check finding exposes reliability risk adjacent to the smell analysis. CPD's negative result is recorded rather than hidden; the manual duplicate-code classification concerns repeated algorithms and policies rather than a long exact clone.

## B4

## 10. Task B4 - Refactoring Strategy

| Smell                   | Fowler-aligned technique                   | Application to TVPSSHub                                                                                                                  |
|-------------------------|--------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------|
| God Class / Large Class | Extract Class; Move Method                 | Create focused RegistrationService/UserAdministrationService (or focused controllers), and move cohesive logic away from UserController. |
| Duplicate Code          | Extract Method; Consolidate Duplicate Code | Introduce a reusable PageSlice<T>/PaginationService and current-user/same-school helper.                                                 |
| Long Method             | Extract Method                             | Extract validation, pagination, model population and authenticated-user lookup.                                                          |
| Dead Code               | Remove Dead Code                           | Delete PMD-confirmed unused imports and rerun PMD.                                                                                       |

### Why these techniques fit

Extract Class creates a new unit around a cohesive responsibility and delegates from the original class, directly reducing reasons to change. Move Method places behavior with the service or policy that owns the required data. Extract Method names and isolates a repeated or conceptually distinct block without changing its sequence. Remove Dead Code deletes elements that cannot affect external behavior.

### Alternative considered

A full rewrite into REST APIs and a client-side frontend was rejected because it changes architecture and creates unnecessary behavioral risk. The recommended approach preserves existing routes, view names, service/DAO calls, and Spring Security annotations while making small, testable movements.

## B4 DESIGN

## 11. Refactoring Design - Extract Class

### Target decomposition

| New/focused component | Moved responsibility                                            | Controller remains responsible for  |
|-----------------------|-----------------------------------------------------------------|-------------------------------------|
| RegistrationService   | Validate uniqueness, encode password, assign default role, save | Binding, error display, redirect    |
| CurrentUserService    | Resolve authenticated user and require login                    | HTTP navigation decision            |
| TeacherUserService    | School-scoped create/edit/delete authorization                  | Route mapping and model             |
| DashboardService      | Aggregate programs, schools, resources                          | Populate view model and return view |

### Proposed interface

```
public interface CurrentUserService {
    UserViewModel requireUser(Authentication authentication);
    void requireSameSchool(UserViewModel actor, UserViewModel target);
}

public interface RegistrationService {
    RegistrationResult register(UserViewModel candidate);
}
```

This design reduces the controller's fan-out and gives authorization/registration logic dedicated unit-test seams. Constructor injection should replace field injection during implementation so required collaborators are explicit and immutable.

## B4 DESIGN

## 12. Refactoring Design - Shared Pagination

### Generic value object

```
public final class PageSlice<T> {
    private final List<T> items;
    private final int currentPage;
    private final int totalPages;
    // constructor and accessors
}

public final class Pagination {
    public static <T> PageSlice<T> slice(List<T> items, int page, int size) {
        int totalPages = Math.max(1, (int) Math.ceil((double) items.size() / size));
        int safePage = Math.max(1, Math.min(page, totalPages));
        int start = Math.min((safePage - 1) * size, items.size());
        int end = Math.min(start + size, items.size());
        return new PageSlice<>(items.subList(start, end), safePage, totalPages);
    }
}
```

### Expected benefit

- One tested definition for empty lists and out-of-range pages.
- Controllers express intent instead of index arithmetic.
- A future page-size or boundary change is made once.
- The extracted component is independent of Spring MVC and easy to unit test.

**BEHAVIOR DECISION** The current handlers differ on empty-list page counts. Before implementation, select and test one external convention (recommended: currentPage=1 and totalPages=1 for an empty UI list) to prevent accidental behavior drift.

## B5 IMPLEMENTATION EVIDENCE

## 13. Task B5 - Before/After Comparison

### Refactoring 1: authenticated-user lookup

| Before                                                                                                                     | After                                                                                         |
|----------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <pre>String email = authentication.getName(); UserViewModel loggedInClient =     userService.findUserByEmail(email);</pre> | <pre>UserViewModel loggedInClient =     currentUserService.requireUser(authentication);</pre> |

### Refactoring 2: pagination

| Before                                                                                                                                         | After                                                                                                                        |
|------------------------------------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------|
| <pre>int totalPages = (int) Math.ceil(...); // clamp page int start = ...; int end = ...; List&lt;T&gt; page = items.subList(start,end);</pre> | <pre>PageSlice&lt;T&gt; page =     Pagination.slice(items, currentPage, 10); // bind page.items/currentPage/totalPages</pre> |

### Refactoring 3: registration workflow

| Before                                                                                                                                | After                                                                                                                                |
|---------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------|
| <p>Controller validates fields and uniqueness, encodes password, assigns role, saves user, and translates persistence exceptions.</p> | <p>Controller delegates registration to RegistrationService and translates a typed RegistrationResult into view errors/redirect.</p> |

**CURRENT STATUS** These are implementation-ready comparisons, not proof of applied repository changes. Replace this page with an annotated git diff after /refactored-code is created and committed.



## B5 VERIFICATION

## 14. Verification Plan and Current Results

### Maven verification transcript

```
> mvn -q test
BUILD RESULT: success (exit code 0)
Important limitation: src/test contains no test classes; this confirms compilation/test
lifecycle only.
> mvn -q -DskipTests pmd:pmd
PMD RESULT: target/pmd.xml generated; 12 violations reported.
> mvn -q -DskipTests pmd:cpd
CPD RESULT: target/cpd.xml generated; no duplication block met the default token threshold.
```

Figure 2. Commands executed against the original repository baseline.

| Feature               | Required post-refactor check                        | Evidence status        |
|-----------------------|-----------------------------------------------------|------------------------|
| Registration          | Valid user, duplicate email/IC, mismatched password | Pending implementation |
| Dashboard             | Authenticated load and unauthenticated redirect     | Pending implementation |
| User pagination       | Empty, first, middle, last and out-of-range pages   | Pending implementation |
| Teacher authorization | Same-school edit/delete; cross-school rejection     | Pending implementation |
| Feedback pagination   | Empty and multi-page activity feedback              | Pending implementation |
| Static analysis       | Rerun PMD/CPD and compare findings                  | Baseline complete      |

A screenshot of the running refactored system cannot be supplied until the refactoring is implemented and the database-backed application is started. The final submission should capture the browser after exercising the affected routes, not merely the login page.

## B5 EVALUATION

## 15. Risk, Regression and Maintainability Assessment

| Risk                                 | Mitigation                                           | Acceptance criterion                              |
|--------------------------------------|------------------------------------------------------|---------------------------------------------------|
| Route/view change                    | Keep annotations and returned view strings unchanged | Existing URLs render same views                   |
| Authorization drift                  | Characterization tests before moving policy logic    | Cross-school access remains denied                |
| Password behavior                    | Keep encoder call and default role assignment        | Stored password encoded; role=3                   |
| Pagination edge cases                | Unit-test empty/out-of-range lists                   | No invalid subList indices                        |
| Spring wiring failure                | Prefer constructor injection; context smoke test     | Application context starts                        |
| New abstractions overcomplicate code | Extract only repeated/cohesive behavior              | Controller dependencies and method sizes decrease |

### Expected improvement summary

The refactoring should lower the size and responsibility count of `UserViewController`, centralize repeated algorithms and authorization policies, and allow focused unit tests without loading MVC infrastructure. New issues are most likely to arise from different empty-page conventions or exception-to-view translations; these must be locked down with tests. Performance is not expected to change materially because the refactoring reorganizes in-process logic without altering database queries.

## FINALIZATION

## 16. Submission Checklist

| Rubric item                 | Current report             | Action before submission                                       |
|-----------------------------|----------------------------|----------------------------------------------------------------|
| B1 description and >500 LOC | Complete                   | Copy baseline to /original-code                                |
| B2 two distinct smells      | Complete                   | Verify line numbers after any baseline edits                   |
| B3 manual + automated       | Complete baseline          | Insert original PMD screenshot if lecturer requires UI capture |
| B4 Fowler techniques        | Complete                   | Keep chosen design aligned with implementation                 |
| B5 applied refactoring      | Not yet complete           | Create /refactored-code and implement changes                  |
| B5 before/after             | Design comparison included | Replace with annotated real diff                               |
| B5 rerun screenshot         | Not available              | Run app and capture affected features                          |
| Commit history              | Not assessed               | Use explicit refactor: ... messages                            |
| PDF minimum 15 pages        | Met by this report         | Recheck after inserting evidence                               |

### Suggested commits

```

refactor: extract pagination into reusable PageSlice utility
refactor: centralize authenticated user and school access checks
refactor: extract registration workflow from UserViewController
refactor: split dashboard and teacher user responsibilities
chore: remove PMD-confirmed unused imports
test: add controller and service characterization tests

```

## SOURCES

## 17. References

1. Fowler, M. Refactoring: Improving the Design of Existing Code (2nd ed.). Addison-Wesley, 2018.
2. Fowler, M. Refactoring Catalog. <https://refactoring.com/catalog/> (accessed 2 July 2026).
3. PMD. PMD Source Code Analyzer Documentation. <https://pmd.github.io/pmd/index.html> (accessed 2 July 2026).
4. PMD. Finding Duplicated Code with CPD. [https://pmd.github.io/pmd/pmd\\_userdocs\\_cpd.html](https://pmd.github.io/pmd/pmd_userdocs_cpd.html) (accessed 2 July 2026).
5. Spring. Spring Boot Reference Documentation 2.7.18. <https://docs.spring.io/spring-boot/docs/2.7.18/reference/htmlsingle/> (accessed 2 July 2026).
6. Project repository files: pom.xml, src/main/java, target/pmd.xml, and target/cpd.xml, reviewed 2 July 2026.

### Evidence integrity note

Figures 1 and 2 are faithful visualizations of machine-generated local outputs and executed commands. They are not screenshots of a refactored running application. The red status notes identify evidence that remains outstanding, preserving academic integrity and making the remaining work explicit.

## APPENDIX

## Appendix A - Full Java SLOC Inventory

| No. | Java source file                                             | SLOC |
|-----|--------------------------------------------------------------|------|
| 1   | src/main/java/com/example/controller/UserViewController.java | 257  |
| 2   | src/main/java/com/example/controller/ActivityController.java | 161  |
| 3   | src/main/java/com/example/dao/UserDAO.java                   | 159  |
| 4   | src/main/java/com/example/model/ActivityViewModel.java       | 144  |
| 5   | src/main/java/com/example/model/School.java                  | 136  |
| 6   | src/main/java/com/example/controller/ResourceController.java | 111  |
| 7   | src/main/java/com/example/dao/ActivityDAO.java               | 105  |
| 8   | src/main/java/com/example/dao/ResourceDAO.java               | 97   |
| 9   | src/main/java/com/example/config/SecurityConfig.java         | 83   |
| 10  | src/main/java/com/example/model/UserViewModel.java           | 78   |
| 11  | src/main/java/com/example/dao/SchoolDAO.java                 | 77   |
| 12  | src/main/java/com/example/model/Resource.java                | 75   |
| 13  | src/main/java/com/example/controller/SchoolController.java   | 64   |
| 14  | src/main/java/com/example/model/Feedback.java                | 59   |
| 15  | src/main/java/com/example/security/CustomUserDetails.java    | 53   |
| 16  | src/main/java/com/example/config/HibernateConfig.java        | 46   |
| 17  | src/main/java/com/example/dao/FeedbackDAO.java               | 45   |
| 18  | src/main/java/com/example/service/UserService.java           | 41   |
| 19  | src/main/java/com/example/service/ActivityService.java       | 39   |
| 20  | src/main/java/com/example/service/ResourceService.java       | 26   |
| 21  | src/main/java/com/example/service/SchoolService.java         | 23   |
| 22  | src/main/java/bdUtil/HibernateCF.java                        | 18   |
| 23  | src/main/java/com/example/controller/HomeController.java     | 10   |
| 24  | src/main/java/com/example/TvpsShubApplication.java           | 9    |

**Total:** 1,916 non-blank, non-comment Java source lines across 24 files.

## APPENDIX

## Appendix B - PMD Finding Register

| File                    | Line | Rule               | Message                                                                                               |
|-------------------------|------|--------------------|-------------------------------------------------------------------------------------------------------|
| ResourceController.java | 106  | MisplacedNullCheck | The null check here is misplaced; if the variable 'auth' is null there will be a NullPointerException |
| UserViewController.java | 13   | UnnecessaryImport  | Unused import 'javax.servlet.http.HttpSession'                                                        |
| UserViewController.java | 14   | UnnecessaryImport  | Unused import 'java.time.LocalDate'                                                                   |
| ActivityDAO.java        | 9    | UnnecessaryImport  | Unused import 'org.hibernate.SessionFactory'                                                          |
| ActivityDAO.java        | 10   | UnnecessaryImport  | Unused import 'org.springframework.beans.factory.annotation.Autowired'                                |
| ActivityDAO.java        | 12   | UnnecessaryImport  | Unused import 'org.hibernate.Hibernate'                                                               |
| FeedbackDAO.java        | 3    | UnnecessaryImport  | Unused import 'com.example.model.ActivityViewModel'                                                   |
| FeedbackDAO.java        | 11   | UnnecessaryImport  | Unused import 'org.hibernate.SessionFactory'                                                          |
| FeedbackDAO.java        | 13   | UnnecessaryImport  | Unused import 'org.springframework.beans.factory.annotation.Autowired'                                |
| ActivityViewModel.java  | 5    | UnnecessaryImport  | Unused import 'java.util.List'                                                                        |
| ActivityViewModel.java  | 6    | UnnecessaryImport  | Unused import 'java.util.ArrayList'                                                                   |
| UserViewModel.java      | 5    | UnnecessaryImport  | Unused import 'org.springframework.format.annotation.DateTimeFormat'                                  |

Generated by PMD 7.17.0. The report file is target/pmd.xml; CPD output is target/cpd.xml.